

RESTAURANT BILLING SYSTEM

Complete Code Documentation

A line-by-line technical walkthrough of
function.h & Project.cpp

Language: C++ | Paradigm: OOP

1. Project Overview

The Restaurant Billing System is a console-based C++ application designed to simulate a simple restaurant ordering workflow. It uses Object-Oriented Programming (OOP) concepts to organize data and behaviour into two main classes: Menu and order.

The project is split across two files:

- `function.h` — Header file containing all class and utility-function definitions.
- `Project.cpp` — Main source file containing the entry point (`main()`) and the interactive program loop.

The system allows a user to: view the menu, place an order by selecting item numbers, and generate a formatted bill showing each ordered item along with the total amount.

2. Header File — `function.h`

The header file is the backbone of the application. It contains the guard directives, necessary includes, a utility function, and two classes.

2.1 Header Guards & Include Directives

Code	Explanation
<code>#ifndef RESTURANT_H</code>	Header guard — prevents the file from being included more than once. If the macro <code>RESTURANT_H</code> is NOT already defined, the code below is compiled.
<code>#define RESTURANT_H</code>	Defines the macro <code>RESTURANT_H</code> so that subsequent inclusions of this file are skipped entirely.
<code>#include <iostream></code>	Includes the standard input/output stream library. Needed for <code>cin</code> (input) and <code>cout</code> (output).
<code>#include <string></code>	Includes the C++ string class, enabling the use of <code>std::string</code> for item names and other text.
<code>#include <iomanip></code>	Includes input/output manipulation utilities. Used here for <code>setw()</code> (set field width) and <code>left</code> (left-align text).
<code>using namespace std;</code>	Brings the entire <code>std</code> namespace into scope so that <code>std::</code> does not need to be written before standard library names.

2.2 Utility Function — `printpadded()`

This standalone function is used to print text with a fixed column width, ensuring that text in multi-column console output lines up neatly.

Code	Explanation
<pre>void printpadded(string text, int width)</pre>	Declares a function named <code>printpadded</code> . It accepts a string (the text to print) and an integer (the column width). It returns nothing (void).
<pre>cout << left</pre>	Sets the output alignment to left-justified for the following output.
<pre><< setw(width)</pre>	Sets the width of the next output field to the value of <code>width</code> . If the text is shorter than <code>width</code> , it is padded with spaces on the right.
<pre><< text;</pre>	Outputs the text string using the alignment and width settings applied above.

2.3 Class — Menu

The `Menu` class holds all the restaurant's food items and their corresponding prices. It exposes a single public method to display them on the console.

2.3.1 Data Members

Code	Explanation
<pre>string itemnames[19] = { ... };</pre>	A fixed-size array of 19 strings, each holding the name of one menu item (e.g., "Burger", "Pizza"). Arrays are zero-indexed, so indices run from 0 to 18.
<pre>float itemprices[19] = { ... };</pre>	A fixed-size array of 19 floats, each storing the price (in Indian Rupees) for the corresponding item. The index matches <code>itemnames</code> (index 0 = Burger = Rs.70).
<pre>int totalitems = 19;</pre>	An integer member that records how many items exist. Used as the loop bound so the code does not need a hard-coded magic number in multiple places.

2.3.2 Method — `displaymenu()`

Prints a neatly formatted menu to the console.

Code	Explanation
------	-------------

<code>void displaymenu()</code>	Declares a public member function with no parameters and no return value.
<code>cout << "===...===" << endl;</code>	Prints a horizontal divider line followed by a newline. Used decoratively above and below the menu heading and at the end.
<code>cout << " MENU ITEMS ";</code>	Prints the centred title of the menu section.
<code>for(int i=0; i<totalitems; i++)</code>	A for-loop that iterates <i>i</i> from 0 up to (but not including) <i>totalitems</i> (19). Each iteration handles one menu item.
<code>cout << i + 1</code>	Prints the item number (1-based, so <i>i</i> +1 is used because arrays are 0-indexed).
<code><< left << setw(26) << itemnames[i]</code>	Prints the item name left-aligned in a 26-character-wide column so prices always start at the same horizontal position.
<code><< "Rs." << itemprices[i]</code>	Prints the price prefix "Rs." followed by the actual price value from the <i>itemprices</i> array.

2.4 Class — order

The order class manages a customer's order within a single session. It stores the items selected and provides methods to take an order and print the final bill.

2.4.1 Private Data Members

Code	Explanation
<code>string ordereditems[50];</code>	A private string array with capacity for 50 items. Stores the name of each item the customer orders.
<code>float orderedprices[50];</code>	A private float array with capacity for 50 items. Stores the price of each corresponding ordered item.
<code>int count;</code>	A private integer that tracks how many items have been ordered so far. Acts as the next-insert index for both arrays.

2.4.2 Constructor

Code	Explanation
<code>order()</code>	Default constructor — called automatically when an order object is created.
<code>{ count = 0; }</code>	Initialises <i>count</i> to 0, ensuring the order starts empty with no items recorded.

2.4.3 Method — *takeorder(Menu m)*

Interactively asks the user to select items from the menu and records each valid selection.

Code	Explanation
<pre>void takeorder(Menu m)</pre>	Accepts a Menu object by value. The menu is passed in so takeorder can display it and access item names/prices.
<pre>m.displaymenu();</pre>	Calls displaymenu() on the passed-in Menu object to show the menu before prompting for input.
<pre>int choice; char more;</pre>	Declares a local integer for the user's item selection and a char to hold the yes/no answer for ordering more.
<pre>do { ... } while (more=='y' more=='Y');</pre>	A do-while loop that always executes at least once. It continues asking for items as long as the user enters 'y' or 'Y'.
<pre>cin >> choice;</pre>	Reads the integer item number entered by the user.
<pre>if (choice >= 1 && choice <= m.totalitems)</pre>	Validates the input — ensures the entered number corresponds to an actual menu item.
<pre>ordereditems[count] = m.itemnames[choice-1];</pre>	Stores the name of the chosen item into the next available slot of ordereditems. choice-1 converts the 1-based user input to a 0-based array index.
<pre>orderedprices[count] = m.itemprices[choice-1];</pre>	Similarly stores the price of the chosen item.
<pre>count++;</pre>	Increments count so the next item is stored in the next array slot.
<pre>else { cout << "Invalid..."; }</pre>	If the user enters a number outside the valid range, an error message is printed and the item is not recorded.
<pre>cin >> more;</pre>	Reads the user's response about whether they want to order more items.

2.4.4 Method — *genrateBill()*

Prints the itemised bill. Note: 'genrate' is a typo for 'generate' in the original source.

Code	Explanation
<pre>void genrateBill()</pre>	Public method with no parameters or return value. Prints the bill to the console.

<code>if (count == 0) { ... return; }</code>	Guard clause — if no items have been ordered, a message is printed and the function exits early (return) to avoid printing an empty bill.
<code>float total = 0;</code>	Local variable to accumulate the running total of all ordered items.
<code>cout << "=== BILL ===";</code>	Prints a decorative header for the bill section.
<code>for(int i=0; i<count; i++)</code>	Loops over every ordered item from index 0 up to count-1.
<code>printpadded(ordereditems[i], 18);</code>	Calls the utility function to print the item name in an 18-character-wide column.
<code>cout << "Rs." << orderedprices[i];</code>	Prints the price for that item on the same line.
<code>total += orderedprices[i];</code>	Adds the item price to the running total.
<code>printpadded("Total", 18);</code>	After the loop, prints the label "Total" using the same 18-character padding.
<code>cout << "Rs." << total;</code>	Prints the computed total amount.
<code>cout << "Thank you...";</code>	Prints a closing thank-you message.

Code	Explanation
<code>#endif</code>	Closes the <code>#ifndef</code> header guard opened at the very top of the file.

3. Main Source File — Project.cpp

Project.cpp contains the entry point of the program. It sets up the interactive menu loop that ties together all the functionality defined in function.h.

3.1 Include Directives & Entry Point

Code	Explanation
<code>#include <iostream></code>	Standard I/O library — required for cin and cout.
<code>#include <string></code>	C++ string class — needed for string variables.
<code>#include <iomanip></code>	I/O manipulation — needed for setw() and left.
<code>#include "function.h"</code>	Includes the project's own header file (quoted, not angled brackets, because it is a user-defined file in the same directory). This makes Menu and order available here.

<code>using namespace std;</code>	Brings the std namespace into scope.
<code>int main()</code>	The entry point of the program. Execution begins here. Returns int (0 = success).

3.2 Local Variable Declarations

Code	Explanation
<code>Menu m;</code>	Creates a Menu object called m. The Menu constructor is the default (compiler-generated) one — all member data is initialised inline in the class definition.
<code>int choice;</code>	Stores the user's main-menu selection (1, 2, 3, or 4).
<code>order Or;</code>	Creates an order object called Or. The order constructor runs, setting count = 0.

3.3 Main do-while Loop

The entire program runs inside a do-while loop, ensuring the menu is shown at least once and keeps repeating until the user selects Exit (4).

Code	Explanation
<code>do { ... } while(choice != 4);</code>	Continues looping as long as choice is not 4 (Exit). The body always executes at least once.
<code>cout << "=== RESTAURANT BILLING SYSTEM ===";</code>	Prints the application banner at the top of each iteration.
<code>cout << "1. View Menu" etc.;</code>	Prints the numbered options for the interactive main menu.
<code>cin >> choice;</code>	Reads the user's chosen option into choice.

3.4 Switch Statement — Case by Case

Code	Explanation
<code>switch(choice)</code>	Evaluates choice and jumps to the matching case label. If no case matches, the default block runs.
<code>case 1:</code>	Triggered when the user enters 1 (View Menu).
<code>m.displaymenu();</code>	Calls the Menu object's displaymenu() method to print all 19 items and prices.

<code>break;</code>	Exits the switch block, preventing fall-through to subsequent cases.
<code>case 2:</code>	Triggered when the user enters 2 (Place Order).
<code>Or.takeorder(m);</code>	Calls <code>takeorder()</code> on the order object, passing the Menu object. The user interactively selects items.
<code>Or.genrateBill();</code>	Immediately generates and prints the bill after the order is placed.
<code>break;</code>	Exits the switch block.
<code>case 3:</code>	Triggered when the user enters 3 (Generate Bill without re-ordering).
<code>Or.genrateBill();</code>	Prints the bill for all items ordered so far in this session.
<code>break;</code>	Exits the switch block.
<code>case 4:</code>	Triggered when the user enters 4 (EXIT).
<code>cout << "Thank you for visiting!";</code>	Prints a farewell message.
<code>break;</code>	Exits the switch block. The while condition (<code>choice != 4</code>) then becomes false, ending the loop.
<code>default:</code>	Catches any invalid input (e.g., 0, 5, 99).
<code>cout << "Invalid choice.";</code>	Informs the user of the invalid input. No break needed as default is the last case.

3.5 Program Exit

Code	Explanation
<code>return 0;</code>	Returns 0 to the operating system, signalling successful program termination.

4. Program Flow Summary

The following summarises the complete execution flow of the program:

1. Program starts → `main()` is called.
2. Objects `m` (Menu) and `Or` (order) are instantiated.
3. The do-while loop begins, displaying the main billing system menu.
4. The user enters a choice (1–4); the switch routes execution to the correct case.
5. Case 1 → `m.displaymenu()` prints the full menu.
6. Case 2 → `Or.takeorder(m)` loops until the user stops; `Or.genrateBill()` prints the bill.

7. Case 3 → Or.genrateBill() prints the current bill without new ordering.
8. Case 4 → farewell message; loop exits.
9. main() returns 0 — program ends.

5. Key OOP Concepts Used

Code	Explanation
Classes & Objects	Menu and order are classes. m and Or are their objects (instances). Each object bundles related data and functions together.
Encapsulation	The order class hides its internal arrays (ordereditems, orderedprices, count) as private. They cannot be accessed directly from outside — only through takeorder() and genrateBill().
Access Modifiers	public: members in Menu are accessible from anywhere. private: members in order protect the internal order state.
Constructor	order() initialises count to 0, guaranteeing a clean state when an order object is created.
Member Functions	displaymenu(), takeorder(), and genrateBill() are methods — functions tied to their respective classes and operating on the class's data.
Passing Objects	takeorder(Menu m) demonstrates passing an object of one class as a parameter to a method of another class.